COMPREHENSIVE GUIDE & STATE OVERVIEW

# Claude Code Advancements

What everything is, how to use it, and where it stands

Session 246 · April 1, 2026 · 41 days old

| | | | |
|---|---|---|---|
| **10,706** TESTS PASSING | **155K** LINES OF CODE | **9** MODULES | **246** SESSIONS |
| **51** MASTER TASKS | **461** INTEL FINDINGS | **18** LIVE HOOKS | **3** CUSTOM AGENTS |

**What is CCA?** A research and development project that builds tools, hooks, agents, and automation for Claude Code — the CLI tool you use every day. CCA makes Claude Code sessions smarter (self-learning, memory, intelligence gathering) and more autonomous (autoloop, multi-chat, desktop automation). It also serves as the R&D brain for the Kalshi prediction market bot, delivering research, analytics, and strategy tools via cross-chat coordination.

github.com/mpshields96/ClaudeCodeAdvancements

CONTENTS

# Table of Contents


1. **The Two Pillars** — What drives everything 3
2. **Nine Modules** — What each does and why it exists 4
3. **Custom Agents (NEW)** — The three agents and how to use them 9
4. **Slash Commands Reference** — Every command with when/where/why 11
5. **Live Hook Infrastructure** — What's running automatically 15
6. **Project Workflows** — How to run CCA for each scenario 16
7. **Recent Discoveries & Upgrades (S234-S246)** — Last 12 sessions 19
8. **Self-Learning System** — How it works in practice 22
9. **Kalshi Cross-Chat Coordination** — The R&D-to-trading-floor pipeline 24
10. **Current State & Next Steps** — Where we are right now 25

THE TWO PILLARS

# The Two Pillars

Everything CCA does maps to exactly two goals. If a task doesn't serve one or both, it shouldn't exist.

**Pillar 1: Get Smarter**
CCA becomes a more capable agent over time. Not by adding gadgets, but by genuinely learning from execution — detecting what works, pruning what doesn't, and applying those learnings automatically in future sessions.

**What this looks like in practice:**
- Session outcomes are tracked, graded, and analyzed for patterns
- Mistakes are auto-captured and resurfaced as warnings in future sessions
- Strategies evolve based on evidence (the YoYo/Sentinel pattern)
- Research findings translate into actionable improvements
- The agent running session N+1 is measurably better than session N

**Key systems:** `self-learning/journal.py`, `reflect.py`, `improver.py`, `principle_registry.py`, `sentinel_bridge.py`, `predictive_recommender.py`, `session_outcome_tracker.py`, `LEARNINGS.md`

**Pillar 2: Get More Bodies**
CCA gains the ability to do more work in parallel. More chats, automated session loops, desktop app control, unattended overnight operation.

**What this looks like in practice:**
- Desktop autoloop runs CCA sessions unattended (you sleep, it works)
- CLI autoloop runs sessions via Terminal
- Hivemind coordination manages multiple parallel chats
- Session daemon auto-spawns chats at optimal times
- Error recovery handles crashes, rate limits, and unexpected states

**Key systems:** `cca_autoloop.py` (116 tests, gate-passed), `desktop_automator.py`, `session_orchestrator.py`, `crash_recovery.py`, `peak_hours.py`

> **How they compound:** A smart agent running 3 parallel sessions knocks out tasks faster than 3 dumb agents or 1 smart agent alone. Pillar 1 makes each session productive. Pillar 2 multiplies the number of sessions. Together they produce exponential progress.

NINE MODULES

# Nine Modules

Each module is a self-contained directory with its own code, tests, and (where applicable) hooks. Here's what each one does, why it exists, and how you interact with it.

### 1. Memory System (`memory-system/` — 340 tests, 1,909 LOC)

**What it does:** Gives Claude a brain that persists across sessions. Captures decisions, preferences, and context so you don't have to re-explain things every time you start a new chat.

**How you use it:**
- It works automatically via PostToolUse and Stop hooks
- When you say "remember X" or "always do Y" or "never do Z", the capture hook extracts it
- CLI viewer: `python3 memory-system/cli.py stats|search|list`
- MCP server available for programmatic queries

**Key components:**
- `hooks/capture_hook.py` — Auto-captures from PostToolUse + Stop events
- `memory_store.py` — SQLite + FTS5 full-text search backend
- `mcp_server.py` — MCP server for external memory queries
- `decay.py` — Ebbinghaus exponential decay (NEW S242) replaces hard TTL cutoffs
  - HIGH confidence: 0.98/day (effectively permanent for active memories)
  - MEDIUM: 0.96/day (50% at 17 days)
  - LOW: 0.93/day (50% at 10 days)

**Relationship to AutoDream:** CCA memory complements Anthropic's native `/dream` feature. Dream does automatic consolidation at session end. CCA memory does structured extraction with typed categories, confidence scoring, and cross-session FTS5 search. They layer, not compete. See `memory-system/DREAM_INTEGRATION_DESIGN.md` for the separation of concerns.

### 2. Spec System (`spec-system/` — 205 tests, 1,191 LOC)

**What it does:** Forces Claude to plan before coding. Instead of diving into code from a vague prompt, you get a reviewable blueprint: requirements, then design, then tasks, then implementation.

**How you use it:** Four slash commands in a strict sequence:
1. `/spec:requirements` — Interview scaffold. Claude asks clarifying questions, produces `requirements.md`
2. `/spec:design` — Architecture generator. Produces `design.md` with mermaid diagrams
3. `/spec:design-review` — Four expert personas (UX, Security, Performance, Maintainability) review the design
4. `/spec:tasks` — Breaks design into atomic, testable tasks ( 100-500 LOC each)
5. `/spec:implement` — Executes one task at a time, commits after each

**Rules:** You must APPROVE each document before the next is generated. No code during requirements or design phases. Each task gets its own commit.

**Key components:**
- `hooks/validate.py` — PreToolUse guard that warns/blocks if you skip phases
- `hooks/skill_activator.py` — Auto-activates on UserPromptSubmit

- `spec_freshness.py` — Detects stale/rotting specs (FRESH/STALE/RETIRED)

### 3. Context Monitor (`context-monitor/` — 434 tests, 3,383 LOC)

**What it does:** Tracks how full Claude's context window is getting and warns you before quality degrades. Prevents the "why did it ignore my rules?" problem that happens when context is silently compacted.

**How you use it:** It runs automatically — no manual interaction needed. You'll see warnings when context gets full, and it blocks exit at critical levels.

**Key components:**
- `hooks/meter.py` — PostToolUse token counter (tracks every tool call)
- `hooks/alert.py` — PreToolUse warn/block at red (70-85%) and critical (85%+)
- `hooks/auto_handoff.py` — Stop hook: blocks exit at critical context levels
- `hooks/compact_anchor.py` — Writes anchors every 10 turns to survive compaction
- `hooks/post_compact.py` — PostCompact recovery (NEW S243): snapshot-specific recovery digest
- `hooks/stop_failure.py` — StopFailure hook: classifies rate limit/auth/server errors
- `session_pacer.py` — Pacing for 2-3h autonomous runs (CONTINUE/WRAP_SOON/WRAP_NOW)
- `session_notifier.py` — Push notifications via ntfy.sh on session end/error
- `auto_wrap.py` — Automatic session wrap trigger

**Zones:** Green (<50%) | Yellow (50-70%) | Red (70-85%) | Critical (85%+)

> **Compaction Protection (NEW S243):** When Claude compacts your context, important state can be lost. CCA now takes a PreCompact snapshot (git status, active tasks, health state) and generates a PostCompact recovery digest. This is wired globally. 81 new tests added in Session 243 (Chat 10).

### 4. Agent Guard (`agent-guard/` — 1,102 tests, 6,964 LOC)

**What it does:** The largest module. Prevents Claude from accidentally deleting files, exposing API keys, or breaking your computer — especially critical during autonomous overnight sessions. Also houses the entire Senior Developer review system (MT-20).

**How you use it:** Mostly automatic via hooks. The Senior Dev review is available on-demand.

**Safety guards (all live in hooks):**
- `credential_guard.py` — Blocks API key / token exposure in any output
- `bash_guard.py` — Blocks dangerous bash commands (rm -rf, kill, network exposure, package installs, evasion via cp/script/dd/tee)
- `path_validator.py` — Blocks dangerous path + system file modifications
- `edit_guard.py` — Prevents Edit retry loops on structured table files (PROJECT_INDEX.md, SESSION_STATE.md)
- `worktree_guard.py` — Worktree isolation guard for Agent Teams (NEW S178/S244)
- `session_guard.py` — Slop detection + commit tracking
- `content_scanner.py` — 9 threat categories for autonomous scanning
- `mobile_approver.py` — iPhone push approval via ntfy.sh for remote sessions

**Senior Developer system (MT-20, 13 modules, 890 tests):**
- `senior_review.py` — On-demand review engine: APPROVE/CONDITIONAL/RETHINK verdicts
- `senior_chat.py` — Interactive CLI chat mode (review + REPL)
- `code_quality_scorer.py` — Aggregate quality scoring (0-100, A-F, 5 dimensions)
- `satd_detector.py` — TODO/FIXME/HACK/WORKAROUND marker detection
- `effort_scorer.py` — PR effort scoring (1-5 scale)
- `coherence_checker.py` — Architectural coherence (module structure, pattern consistency)
- `fp_filter.py` — False positive filter (test files, vendored code, low-confidence)

- `adr_reader.py` — ADR discovery + relevance matching
- `git_context.py` — File history, blame ownership, churn detection

### 5. Usage Dashboard (`usage-dashboard/` — 369 tests, 3,030 LOC)

**What it does:** Token and cost transparency. Shows exactly how many tokens and dollars each session costs, so you can spot waste and stay within Max subscription limits.

**How you use it:**
- `/arewedone` — Structural completeness checker (are all tests green? all docs current?)
- Cost alerts fire automatically via PreToolUse hook at configurable thresholds
- `doc_drift_checker.py` — Verifies docs match reality (test counts, file paths)
- `hook_profiler.py` — Measures hook chain latency (are hooks slowing things down?)

**Key components:**
- `usage_counter.py` — CLI token/cost counter
- `otel_receiver.py` — OpenTelemetry HTTP/JSON receiver
- `hooks/cost_alert.py` — PreToolUse cost threshold alerts

### 6. Reddit Intelligence (`reddit-intelligence/` — 498 tests, 4,964 LOC)

**What it does:** Automatically finds the best new tools, tips, and techniques from Reddit so you don't have to scroll through hundreds of posts yourself. Also scans GitHub trending repos.

**How you use it:**
- `/cca-review <url>` — Review any URL against the five frontiers (automatic on URL paste)
- `/cca-nuclear` — Deep-dive batch review (autonomous, 100-150 posts)
- `/cca-nuclear-daily` — Daily hot+rising intelligence scan
- `/cca-nuclear-github` — GitHub trending repo intelligence
- `/cca-scout` — Scan subreddits for high-signal posts
- `/reddit-research` — Deep Reddit research dive on a topic
- `/reddit-intel:ri-scan` — Subreddit intelligence scanner
- `/reddit-intel:ri-read` — Read a specific Reddit URL

**Key components:**
- `reddit_reader.py` — Fetches posts + ALL comments (best insights are buried 3-4 levels deep)
- `autonomous_scanner.py` — Full scan pipeline with prioritizer + safety
- `github_scanner.py` — GitHub repo intelligence + domain-based discovery
- `profiles.py` — 10 builtin subreddit profiles across 4 domains (claude/trading/dev/research)
- `subreddit_discoverer.py` — Domain-based subreddit discovery

**Verdicts:** BUILD / ADAPT / REFERENCE / REFERENCE-PERSONAL / SKIP

**FINDINGS_LOG.md:** Append-only log of all 461 intelligence findings. Each entry has a verdict, frontier mapping, rat poison check, and actionable takeaway. This is CCA's institutional knowledge.

### 7. Self-Learning (`self-learning/` — 2,391 tests, 30,981 LOC)

**What it does:** The largest module by far. Makes the Kalshi bot and CCA itself smarter over time by analyzing what worked and what didn't. Also houses the UBER investment research engine (MT-37) and Kalshi-specific analytics tools.

**How it works (the loop):**
1. Events logged to `journal.jsonl` (1,226 events: wins, pains, outcomes, feedback)
2. `reflect.py` detects patterns across events
3. `improver.py` proposes strategy changes (YoYo evolution loop)
4. `principle_registry.py` scores what works across domains (Laplace scoring)

5. `sentinel_bridge.py` adds 20% adaptive mutation (analyze failures, generate counter-strategies)
6. `session_outcome_tracker.py` tracks prompt-to-outcome at session level

**Kalshi-specific tools:**
- `trade_reflector.py` — 5 pattern detectors on trade history
- `calibration_bias.py` — Systematic mispricing detection
- `dynamic_kelly.py` — Per-bucket optimal sizing
- `strategy_health_scorer.py` — HEALTHY/MONITOR/PAUSE/KILL verdicts
- `monte_carlo_simulator.py` — Ruin probability + bankroll trajectory
- `sizing_optimizer.py` — Kelly fractions, daily EV/SD, P(target)
- `daily_outlook.py` — BTC range to volume to EV/SD to verdict
- `hour_sizing.py` — Time-of-day sizing adjuster

**UBER Investment Research (MT-37, 12 modules, 250 tests):**
- `market_data.py` — Returns, volatility, beta, factor exposures
- `allocation.py` — Equal-weight, risk parity, Black-Litterman
- `factor_tilts.py` — Value/momentum/quality/low-vol overlays
- `kelly_sizer.py` — Fractional Kelly criterion with confidence scaling
- `risk_monitor.py` — Drawdown tracker, rolling volatility, risk dashboard
- `tax_harvester.py` — TLH scanner with wash sale tracking
- `withdrawal_planner.py` — CAPE-adjusted safe withdrawal rates
- `rebalance_advisor.py` — Hybrid threshold + calendar rebalancing
- `portfolio_report.py` — Sharpe/Sortino/drawdown/risk attribution
- `behavioral_guard.py` — 5 behavioral bias detectors
- `uber_pipeline.py` — Unified orchestrator for all modules
- `dca_advisor.py` — Dollar-cost averaging for $20/week recurring investing

**Mistake-Learning (NEW S241):**
- `correction_detector.py` — Detects error-then-correction sequences
- `hooks/correction_capture.py` — Auto-captures corrections to journal
- `hooks/failure_capture.py` — PostToolUseFailure feeds into correction pipeline
- `resurfacer.py corrections` — Surfaces recent corrections at session init (wired in /cca-init Step 2.97)

### 8. Design Skills (`design-skills/` — 1,604 tests, 12,765 LOC)

**What it does:** Generates professional visual output — PDF reports, HTML dashboards, presentation slides, SVG charts, and website pages. All from one command, no manual formatting.

**How you use it:**
- `/cca-report` — Generate comprehensive PDF status report (this report uses it)
- `/cca-dashboard` — Generate interactive HTML dashboard (dark mode, sortable, searchable)
- `/cca-slides` — Generate 16:9 presentation slides
- `/cca-website` — Generate landing page + docs page HTML
- `/cca-status` — Nuclear-level project overview

**Key components:**
- `report_generator.py` — Data collector + Typst renderer CLI
- `chart_generator.py` — 29 SVG chart types (bar, line, donut, heatmap, scatter, box, histogram, candlestick, forest, calibration, bullet, slope, lollipop, dumbbell, pareto, gauge)
- `report_charts.py` — 29 report-specific charts (13 base + 13 Kalshi + 3 learning)
- `dashboard_generator.py` — Self-contained HTML dashboard
- `chartjs_bridge.py` — Chart.js config generator for interactive dashboards (8 types)
- `kalshi_data_collector.py` — Read-only Kalshi bot DB analytics (8 chart methods)
- `learning_data_collector.py` — Self-learning intelligence (3 chart methods)

- `report_differ.py` — Structured diff between two report sidecars (test growth, MT transitions)
- `figure_generator.py` — Multi-panel figure composer (grid layout, panel labels, captions)
- `daily_snapshot.py` — Daily project metric snapshots with diff support

### 9. Research (`research/` — 86 tests, 1,397 LOC)

**What it does:** R&D sandbox for experimental features. Things get prototyped here before becoming real modules.

**Current contents:**

- iOS project generator (SwiftUI)
- Xcode build wrapper
- `MT53_EMULATOR_RESEARCH.md` — Emulator comparison for Pokemon bot
- `MT53_INTELLIGENCE_SCAN.md` — 14 Pokemon AI projects analyzed
- `AGENT_ORCHESTRATION_RESEARCH.md` — 8 frameworks, 3 papers, 4 gaps

CUSTOM AGENTS

# Custom Agents (NEW)

Claude Code now supports custom agents via `.claude/agents/` directory. These are markdown files with frontmatter that define specialized sub-agents with their own model, tool restrictions, and behavior. CCA has three custom agents, all built and validated in Sessions 245-246 (March 31, 2026).

> **What are custom agents?** They're specialized Claude instances spawned as sub-processes. Each has a specific model (haiku for cheap tasks, sonnet for analysis, opus for deep review), restricted tools (can't edit files, can't spawn more agents), and a focused prompt. The parent Claude orchestrates them via the `Agent` tool with `subagent_type` parameter.

### Agent 1: `cca-test-runner` (Haiku)

**File:** `.claude/agents/cca-test-runner.md`

**What it does:** Runs CCA's test suites and reports results. Nothing else.

**When to use it:** After code changes, when you need test verification without burning expensive Opus tokens. Haiku is 20x cheaper than Opus.

**How to invoke it:**

```
Agent tool → subagent_type: "cca-test-runner"
```

Or Claude will use it automatically after code changes during `/cca-auto`.

**Configuration:**
- Model: `haiku` (cheapest, fast)
- Max turns: 10 (test runs shouldn't need more)
- Disallowed tools: Edit, Write, Agent, WebFetch, WebSearch (read-only)

**Output format:**

```
RESULT: 350/350 suites passed, 12199 tests total, 35.6s elapsed
```

If failures exist, they're listed individually before the summary.

**Three modes:**
1. Full suite: `python3 parallel_test_runner.py --workers 8`
2. Quick smoke (10 critical suites): `--quick`
3. Changed files only: `--changed-only`

### Agent 2: `cca-reviewer` (Sonnet)

**File:** `.claude/agents/cca-reviewer.md`

**What it does:** Reviews any URL (Reddit, GitHub, blog, docs) against CCA's five frontiers and delivers a BUILD/ADAPT/REFERENCE/SKIP verdict. This is the autonomous intelligence gathering engine.

**When to use it:** When you paste any URL into the chat. Claude PROACTIVELY spawns this agent — you don't need to ask. Just paste a link and say "check this out."

**How it works:**

1. For Reddit URLs: runs `reddit_reader.py` to get post + all comments
2. For GitHub URLs: uses WebFetch to read README, structure, key files
3. For any other URL: uses WebFetch to read full page content
4. Follows links found in comments to related repos/tools
5. Evaluates against all 5 frontiers + rat poison check

**Verdict format:**

```
REVIEW: [title]
Source: [URL]
Score: [N] pts | [N]% upvoted | [N] comments

FRONTIER: [1-5 name] or NEW or OFF-SCOPE
RAT POISON: CLEAN / CONTAMINATED
WHAT IT IS: [2-3 sentences]
WHAT WE CAN STEAL: [specific patterns worth taking]
IMPLEMENTATION: delivery vehicle, effort, dependencies
VERDICT: BUILD / ADAPT / REFERENCE / REFERENCE-PERSONAL / SKIP
WHY: [justification]
```

**Configuration:**
- Model: `sonnet` (balanced cost/quality for analysis)
- Max turns: 30 (needs time to read deeply)
- Disallowed tools: Edit, Write, Agent (read-only)
- Effort: high
- Color: cyan (visual indicator in output)

**Key features:**
- "Vocabulary payload" in the prompt pre-loads CC terminology so Sonnet understands the domain
- Anti-rubber-stamp: "Do NOT be generous. Most things are REFERENCE or SKIP."
- REFERENCE-PERSONAL is valid for tools useful to you personally (trading, academic writing)

### Agent 3: `senior-reviewer` (Opus)

**File:** `.claude/agents/senior-reviewer.md`

**What it does:** Performs senior developer code review. Read-only — it CANNOT modify files. This forces it to clearly articulate what's wrong instead of just fixing it (better feedback).

**When to use it:** After significant code changes, or manually via `/senior-review`.

**How to invoke it:**

```
/senior-review
Agent tool with subagent_type: "senior-reviewer"
```

**Verdict format:**

```
SENIOR REVIEW — [filename]
Verdict: APPROVE / CONDITIONAL / RETHINK

Metrics:
  LOC: [n]  |  Quality: [grade] ([score]/100)  |  Effort: [label] ([n]/5)
  SATD: [n] markers ([n] HIGH)  |  Blast radius: [n] dependents

Issues:
1. [specific issue with line number, what's wrong, why it matters]

Senior take: [2-3 sentences as a senior dev would speak]
```

**Configuration:**

- Model: `opus` (needs the best model for deep analysis)
- Max turns: 15
- Disallowed tools: Edit, Write, Agent, WebFetch, WebSearch (read-only)
- Effort: high
- Color: magenta

**Anti-rubber-stamp rule:** MANDATORY — must identify at least ONE issue or improvement, even for APPROVE verdicts. Zero-issue reviews are rubber stamps. This was validated in S246 (Chat 14.5): the agent found 5 real issues in a CONDITIONAL verdict, confirming anti-rubber-stamp works.

**Two-step process:**
1. Runs the automated review engine (`senior_review.py` — SATD, effort, quality scoring)
2. Performs manual deep analysis (correctness, security, maintainability, test coverage, architecture)
3. Checks up to 3 imported modules to verify interface contracts

SLASH COMMANDS

# Slash Commands Reference

Every CCA slash command lives in `.claude/commands/`. Here's each one with practical context on when, where, and why you'd use it.

### Session Lifecycle

**/cca-init**
Session startup. Reads context, runs smoke tests, shows briefing. Run once at start.

**When:** Beginning of every CCA session. First thing you type. **What it does:** Reads SESSION_STATE.md + PROJECT_INDEX.md + CLAUDE.md, runs 10-suite smoke test ( 2 seconds via slim_init.py), checks cross-chat comms, shows priority picker recommendation, surfaces recent corrections as warnings, and outputs a briefing with next work item. **Slim mode:** Default since S99b. 74% faster than full init. Falls through to full mode only if smoke tests fail.

**/cca-auto**
Autonomous work mode. Picks tasks, executes with TDD, chains continuously.

**When:** After `/cca-init`, when you want Claude to work autonomously. **What it does:** Reads TODAYS_TASKS.md first (authoritative), falls through to priority_picker if all TODOs complete. Executes tasks with test-driven development. Commits after each task. Chains to next task automatically. Monitors context health via session_pacer.py. **Budget awareness:** Checks peak/off-peak hours. During peak (8AM-2PM ET weekdays): 40-50% budget, no agent spawns. Off-peak: 100% budget, full autonomy.

**/cca-wrap**
Session wrap-up. Self-grades, updates all docs, generates resume prompt.

**When:** End of every session (automatic when context reaches wrap threshold, or manual). **What it does:** 10 steps: commit uncommitted work, run full test suite, generate self-grade, update SESSION_STATE.md + PROJECT_INDEX.md, run batch_wrap_analysis.py (7 analyses in one call), write self-learning journal entries, cross-chat delivery if applicable, generate SESSION_RESUME.md, fire autoloop trigger (Step 10 — NEVER skip this).

### Dual-Chat Mode

When running two CCA chats simultaneously (Desktop + CLI worker):

**/cca-desktop**
Combined launcher for desktop coordinator. Init + auto in one command.

**When:** You want to run the desktop coordinator chat (owns shared docs). **What it does:** Sets `CCA_CHAT_ID=desktop`, runs init, enters auto mode. Owns SESSION_STATE.md, PROJECT_INDEX.md, CHANGELOG.md. Assigns tasks to workers via `cca_comm.py`.

**/cca-worker**
Combined launcher for CLI worker. Init + auto in one command.

**When:** You want to run a worker chat (executes assigned tasks). **What it does:** Sets `CCA_CHAT_ID=cli1`, checks inbox via `cca_comm.py inbox`, claims scope, executes assigned task, reports back via `cca_comm.py done`. Never modifies shared docs.

**/cca-wrap-desktop / /cca-wrap-worker**
Role-specific wrap commands.

**When:** Wrapping dual-chat sessions. Desktop wraps docs. Worker wraps code only.

### Intelligence & Research

**/cca-review <url>**
Review any URL against five frontiers. BUILD/SKIP verdict.

**When:** You paste any URL — Reddit, GitHub, blog, docs. Claude auto-triggers this. **What it does:** Spawns `cca-reviewer` agent. Reads everything (all Reddit comments, full README). Delivers frontier mapping, rat poison check, implementation feasibility, and verdict.

**/cca-scout**
Scan subreddits for high-signal posts.

**When:** You want a quick scan of what's hot across Claude Code subreddits. **What it does:** Scans configured subreddit profiles, triages by title, deep-reads top candidates. Faster than `/cca-nuclear` (5-10 min vs 45-60 min).

**/cca-nuclear**
Deep-dive batch review. Autonomous. 100-150 posts.

**When:** Dedicated intelligence gathering session. Off-peak hours preferred (heavy token usage). **What it does:** Fetches top posts from multiple subreddits, reads ALL of them including comments, delivers verdicts, updates FINDINGS_LOG.md. The "nuclear" scan.

**/cca-nuclear-daily**
Daily hot+rising intelligence scan.

**When:** Daily routine. Lighter than full nuclear. Focuses on today's hot/rising posts. **Includes Phase 0 discovery:** Runs `subreddit_discoverer.py` to find new relevant subreddits.

**/cca-nuclear-github**
GitHub trending repo intelligence.

**When:** Scanning GitHub for interesting repos. Separate from Reddit scanning. **Includes Phase 0 discovery:** Runs `github_scanner.py discover` to find new repos by domain.

**/cca-nuclear-wrap**
Wrap after a nuclear scan session.

**When:** After completing a nuclear scan. Updates findings, commits, generates resume.

**/reddit-research**
Deep Reddit research dive on a specific topic.

**When:** You want focused research on one topic across Reddit. More targeted than nuclear.

**/reddit-intel:ri-scan**
Subreddit intelligence scanner.

**/reddit-intel:ri-read <url>**
Read a specific Reddit URL (full post + all comments).

**/reddit-intel:ri-loop**
Schedule weekly Reddit scans.

### Output & Reporting

**/cca-report**
Generate comprehensive PDF status report. The report you're reading now.

**When:** End of day, or when you want a snapshot of project state. **What it does:** Collects data from all modules, parses master tasks, generates 32 SVG charts, renders a 24-page Typst PDF with executive summary, module deep-dives, master task status, hook infrastructure, intelligence stats, architecture decisions, and risks. **Output:** `CCA_STATUS_REPORT_YYYY-MM-DD.pdf` + JSON sidecar for diffing against previous reports.

**/cca-dashboard**
Generate interactive HTML dashboard.

**When:** You want a browsable, interactive view. Dark mode, sortable tables, collapsible sections. **Output:** Self-contained HTML file. No server needed — just open in browser.

**/cca-slides**
Generate 16:9 presentation slides (PDF).

**When:** You need to present CCA progress. Professional slide deck.

**/cca-website**
Generate landing page + docs page (HTML).

**When:** You want a public-facing page for the project.

**/cca-status**
Nuclear-level project overview.

**When:** Quick comprehensive status check. Less formal than `/cca-report`.

**/arewedone**
Structural completeness check.

**When:** Before declaring something done. Checks: all tests green? All docs current? Any stubs?

### Dev Tools

**/senior-review**
On-demand senior developer code review.

**When:** After significant code changes. Spawns `senior-reviewer` agent (Opus). **Delivers:** APPROVE/CONDITIONAL/RETHINK verdict with specific issues and line numbers.

**/spec:requirements → /spec:design → /spec:design-review → /spec:tasks → /spec:implement**
Spec-driven development pipeline.

**When:** Building a new feature that needs planning. Strict sequence, approval gates.

**/ag-ownership**
File ownership manifest.

**When:** Checking which module owns which files. Used by hivemind to prevent conflicts.

**/handoff**
Session handoff (MEM-4).

**When:** Passing work to another chat mid-session.

**/browse-url <url>**
Read any URL content.

**When:** You just want the content without the review analysis.

### Pokemon Bot

**/pokemon-play**
Play Pokemon Crystal via the autonomous bot (MT-53).

**When:** Running the Pokemon Crystal bot. Uses mGBA emulator (PyBoy is BANNED). **Status:** 35% complete. Emulator control, memory reading, battle AI, boot sequence built. Needs mGBA backend migration.

LIVE HOOKS

# Live Hook Infrastructure

CCA has 18 hooks wired globally in `~/.claude/settings.local.json` and project-level `.claude/settings.local.json`. These run automatically on every relevant event — zero cost, deterministic, no token usage.

### PreToolUse (2 hooks — run BEFORE each tool call)

| Hook | What it does |
|---|---|
| `context alert` | Warns at red (70-85%), blocks expensive tools at critical (85%+) |
| `path validator` | Blocks writes to dangerous paths (/etc, /System, /.ssh) |

### PostToolUse (1 hook — runs AFTER each tool call)

| Hook | What it does |
|---|---|
| `loop detector` | Catches agents stuck in repetitive cycles (3+ similar outputs, 0.80 threshold) |

### UserPromptSubmit (1 hook — runs on your input)

| Hook | What it does |
|---|---|
| `spec activator` | Auto-activates spec-system skills when relevant keywords detected |

### Stop (1 hook — runs when Claude finishes responding)

| Hook | What it does |
|---|---|
| `memory capture` | Extracts and persists explicit memory triggers (“remember”, “always”, “never”) |

### PreCompact (1 hook — runs BEFORE context compaction)

| Hook | What it does |
|---|---|
| `compact snapshot` | Captures git status, active tasks, health state before compaction destroys them |

### PostCompact (1 hook — runs AFTER context compaction)

| Hook | What it does |
|---|---|
| `compact recovery` | Generates snapshot-specific recovery digest with what was lost |

### PostToolUseFailure (wired globally)

| Hook | What it does |
|---|---|
| `failure capture` | Feeds tool failures into correction pipeline for mistake-learning |

### StopFailure (wired globally)

| Hook | What it does |
|---|---|
| `stop failure` | Classifies rate limit / auth / server errors. Tracks failure state. |

> **Why hooks over Auto Mode?** Anthropic's Auto Mode uses a Sonnet classifier before each tool call — it costs tokens on every action. CCA's hooks are rule-based Python scripts: zero cost, deterministic, and don't burn any tokens. Community consensus (r/ClaudeCode, 479pts): power users prefer custom hooks over Auto Mode. Our approach is validated.

PROJECT WORKFLOWS

# Project Workflows

Here's how to run CCA for each scenario you encounter.

**Workflow 1: Standard CCA Session (Solo)**
The default. One chat, autonomous work.

**Step 1:** Open a Terminal tab
**Step 2:** `cd ~/Projects/ClaudeCodeAdvancements && claude --model claude-opus-4-6`
**Step 3:** Type `/cca-init` — reads context, runs smoke tests, shows briefing
**Step 4:** Type `/cca-auto` — autonomous work begins
**Step 5:** Claude works through TODAYS_TASKS.md items, then priority_picker
**Step 6:** Session wraps automatically when context reaches threshold, or type `/cca-wrap`
**Step 7:** Autoloop trigger fires (Step 10) — spawns next session if configured

**Budget:** Peak hours (8AM-2PM ET): 40-50% budget, no agent spawns. Off-peak: full power.

**Workflow 2: Dual-Chat Mode (Desktop + Worker)**
Two parallel CCA chats. Desktop coordinates, worker executes.

**Chat 1 (Desktop coordinator):**
→ Type `/cca-desktop` — inits, assigns tasks to worker, owns shared docs
→ Manages SESSION_STATE.md, PROJECT_INDEX.md, CHANGELOG.md
→ Queues tasks via `cca_comm.py say cli1 "task description"`
→ Wraps with `/cca-wrap-desktop`

**Chat 2 (CLI worker):**
→ Type `/cca-worker` — inits, checks inbox, claims scope, executes
→ `python3 cca_comm.py inbox` to receive tasks
→ `python3 cca_comm.py claim "module-name"` before touching files
→ `python3 cca_comm.py done "summary"` when finished
→ Wraps with `/cca-wrap-worker`

**When to use:** 6+ independent tasks. 2-chat pattern proven more reliable than 3-chat. Each chat stays at 50% budget (combined 100%).

**Workflow 3: CCA + Kalshi (Standard 3-Chat)**
The typical daily configuration. CCA works on its tasks while Kalshi bot runs.

**Chat 1:** `cd ~/Projects/polymarket-bot && claude` → `/kalshi-main`
→ Kalshi bot monitoring + trading execution

**Chat 2:** `cd ~/Projects/polymarket-bot && claude` → `/kalshi-research`
→ Kalshi research + edge discovery

**Chat 3:** `cd ~/Projects/ClaudeCodeAdvancements && claude --model claude-opus-4-6`
→ `/cca-init` then `/cca-auto`
→ CCA autonomous work, checks cross-chat comms every cycle

**Cross-chat coordination:**
- CCA reads `~/.claude/cross-chat/POLYBOT_TO_CCA.md` for Kalshi requests
- CCA writes `~/.claude/cross-chat/CCA_TO_POLYBOT.md` for deliveries
- Kalshi main checks CCA deliveries every 3rd monitoring cycle ( 15 min)
- Deliveries must be IMMEDIATELY actionable: specific functions, thresholds, data fields

**Workflow 4: Overnight Autonomous**
Exactly 3 chats. No duplicates. No permission prompts.

**Chat 1:** Kalshi main (monitoring + trading)
**Chat 2:** Kalshi research (edge discovery)
**Chat 3:** CCA autonomous (one chat only — never duplicate)

**Key rules:**
→ 90% budget (off-peak, no 2x promo anymore)
→ No permission prompts (hooks handle safety)
→ Proper wrap + autoloop at session end
→ Never close Kalshi windows

**Workflow 5: Intelligence Gathering Session**
Dedicated to scanning Reddit/GitHub for tools and research.

**Quick scan:** `/cca-scout` — 5-10 minutes, title-first triage
**Full scan:** `/cca-nuclear` — 45-60 minutes, reads everything
**Daily routine:** `/cca-nuclear-daily` — hot + rising posts
**GitHub focus:** `/cca-nuclear-github` — trending repos
**Specific URL:** Paste link → auto-review (cca-reviewer agent)
**Deep research:** `/reddit-research <topic>` — focused dive

**All findings logged to FINDINGS_LOG.md** (461 entries and counting). Each gets a verdict (BUILD/ADAPT/REFERENCE/SKIP) and frontier mapping.

**Workflow 6: Feature Development (Spec-Driven)**
For new features that need planning. Strict sequence.

**Step 1:** `/spec:requirements` — Interview. Produces requirements.md. You approve.
**Step 2:** `/spec:design` — Architecture + mermaid diagrams. You approve.
**Step 3:** `/spec:design-review` — 4 expert personas review the design. You approve.
**Step 4:** `/spec:tasks` — Atomic task breakdown. You approve.
**Step 5:** `/spec:implement` — One task at a time, commit after each.
**Step 6:** `/senior-review` — Senior dev reviews the result.

**Rules:** No code during requirements or design phases. Each document needs explicit approval before the next is generated. Each implementation task gets its own commit.

### Workflow 7: Pokemon Crystal Bot (MT-53)

**Status:** 35% complete. Emulator control, memory reading, battle AI, boot sequence built.

**To play:** `/pokemon-play`
**Emulator:** mGBA (PyBoy is BANNED — Matthew directive S219)
**Philosophy:** Run first, build while playing (S219 directive)
→ Get emulator running → Validate boot + RAM reading (–offline, no API cost)
→ Connect LLM and let it play → Fix what breaks → Build features while playing

**Key files:** `pokemon-agent/agent.py` (989 LOC), `emulator_control.py`, `memory_reader.py`, `crystal_data.py` (251 species/moves), `battle_ai.py`, `boot_sequence_crystal.py`
**Reference repos:** `pokemon-agent/references/` (cloned for code theft per S218 directive)

RECENT DISCOVERIES

# Recent Discoveries & Upgrades (S234-S246)

The last 12 sessions (March 30 - March 31) were an intensive sprint. Here's what was built, discovered, and upgraded.

**Phase 1: Reddit Intelligence Batch (S234-S235)**
**What happened:** Matthew dropped 21 Reddit links for review. Two full sessions of reading every post and all comments. Critical discoveries emerged.

**Key finds:**
- **Cache bugs PSA (454pts):** Two bugs silently 10-20x API costs on CC v2.1.69+. Bug 2 affects us. Every –resume causes full cache miss. Cost: $0.04-0.15/request at 500K context. Workaround: prefer fresh sessions over –resume.
- **1M context window burns limits (155pts):** Cache TTL 1hr Max vs 5min Pro. 15-40% optimal context zone. `CLAUDE_CODE_DISABLE_1M_CONTEXT=1` env var applied.
- **Post-promo rate limits WORSE than pre-promo:** Universal community concern. New 100% is  80-90% of old baseline.
- **CC source leaked (1000pts):** Full TypeScript codebase. KAIROS (persistent memory + dream), Coordinator Mode, UDS Inbox (Unix socket comms), Daemon Mode. Validates CCA research.

**Phase 2: Build & Harden (S236-S241, Chats 1-7)**
**Loop Detection Guard (S238):**
- `agent-guard/loop_detector.py` + `hooks/loop_guard.py` — catches autonomous agents stuck in repetitive cycles. 40 tests. Wired globally as PostToolUse hook.
- Configurable threshold (default 0.80 similarity). Env var `CLAUDE_LOOP_THRESHOLD`.

**Wrap Optimization (S240):**
- `batch_wrap_analysis.py` — consolidates 7 wrap steps into 1 Python script. 18 tests.
- Wrap command trimmed 61% (601 ⟶ 244 lines). Full docs moved to WRAP_REFERENCE.md.
- Conditional cross-chat skip saves  5K tokens when no Kalshi work done.
- **Result:** Wrap cost dropped from  20K to  12K tokens (40% reduction).

**Dream Integration Design (S239):**
- `memory-system/DREAM_INTEGRATION_DESIGN.md` — maps exactly how CCA memory layers on top of Anthropic's native AutoDream. CCA does structured extraction with types/TTL/confidence. Dream does automatic consolidation. They complement, don't compete.

**Mistake-Learning Pattern (S241):**
- `correction_detector.py` — detects error⟶correction sequences in tool output
- `hooks/correction_capture.py` — auto-captures corrections to journal (68 tests)
- Ported to Kalshi project settings for cross-project benefit
- Wired into /cca-init as Step 2.97 for automatic resurfacing at session start

**Phase 3: Deeper Patterns (S242-S243, Chats 8-10)**
**Ebbinghaus Memory Decay (S242):**
- `memory-system/decay.py` — replaces hard TTL cutoffs with smooth exponential decay
- Per-confidence rates: HIGH=0.98/day, MEDIUM=0.96/day, LOW=0.93/day

- A memory at day 89 (LOW) no longer vanishes at day 91 — it fades gracefully
- 33 tests. Integration pending (needs `last_accessed_at` schema field).

**Correction Resurfacing (S242):**
- Added `get_recent_corrections()`, `format_correction_warnings()` to resurfacer.py
- Wired into /cca-init Step 2.97 — corrections auto-surface at session start
- `python3 self-learning/resurfacer.py corrections --days 7` CLI command
- 34 new tests.

**Compaction Protection (S243):**
- CTX-8 PreCompact hook captures git/task/health state before compaction
- Enhanced PostCompact generates snapshot-specific recovery digest
- 81 new tests, 488 context-monitor total. Wired globally.
- **Why this matters:** When Claude compacts context, instructions from early in the session can be lost. This captures the critical state before compaction and restores it after.

**Phase 4: Architecture & Agents (S244-S246, Chats 11-14.5)**
**CC Feature Exploration (S244):**
- `CC_FEATURE_NOTES.md` — 16 agent frontmatter fields mapped (model, maxTurns, disallowedTools, effort, color, allowedTools, rollout, etc.)
- Custom agent design specs for all 3 agents
- `AGENT_TEAMS_VS_HIVEMIND.md` — COMPLEMENT verdict (native agent teams for simple tasks, hivemind for complex multi-session coordination)

**CC Source Study (S245):**
- Cloned real CC TypeScript source (1,892 files)
- Coordinator Mode analysis: 3-layer transport (REPL ⟶ local-stdio ⟶ UDS/WebSocket)
- Compaction pipeline fully documented (the exact bug that causes state loss)
- CLAUDE.md 11.4K token audit with 58% reduction plan
- 10 derivative repos mapped to CCA frontiers

**10 Principles Research (S245):**
- PRISM identities: brief identities (<50 tokens) outperform elaborate ones
- 45% threshold: single agent >45% optimal = don't add more agents
- Rubber-stamp prevention: mandatory minimum-one-issue rule
- Lost-in-middle: >30% accuracy drop beyond middle of context (Liu 2024)
- 5-agent team = 7x tokens for 3.1x output (DeepMind 2025)

**First Custom Agents (S245):**
- `cca-test-runner` (haiku) built and validated — cheap test execution
- `cca-reviewer` (sonnet) built with vocabulary routing — intelligent URL review

**Senior Reviewer + Validation (S246):**
- `senior-reviewer` (opus) built with anti-rubber-stamp rule
- All 3 agents validated end-to-end
- DMCA preservation: cloned claw-code and claude-code-source-build to references/
- Cache audit: 68-99% cache read ratios confirmed healthy

SELF-LEARNING

# Self-Learning System

The self-learning system is CCA's compound interest machine. Here's how it actually works in practice, not just in theory.

### The Loop

**1. Capture** → Events logged to `journal.jsonl` (1,226 events total)
Types: session_outcome, win, pain, feedback_roi, sentinel_stats, meta_learning_health

**2. Detect** → `reflect.py` finds patterns across events
"We keep making the same mistake" → escalate to LEARNINGS.md
"This approach worked 3 sessions in a row" → reinforce in strategy.json

**3. Evolve** → `improver.py` proposes changes (YoYo pattern)
Small mutations to strategy parameters. If outcomes improve, keep. If not, revert.

**4. Score** → `principle_registry.py` ranks what works (Laplace scoring)
Cross-domain principle scoring with confidence modifiers.

**5. Mutate** → `sentinel_bridge.py` adds 20% adaptive variation
Analyze failures → generate counter-strategies → cross-pollinate across domains.

**6. Resurface** → At session start, relevant findings + corrections are shown
You see warnings about past mistakes before you repeat them.

### What Gets Captured

- **Session outcomes:** 192 events tracking what each session produced, its grade, and whether the resume prompt was followed
- **Wins:** 257 events recording what went right (specific patterns that worked)
- **Pains:** 137 events recording what went wrong (specific failure modes)
- **Corrections:** Error→fix sequences captured automatically via PostToolUseFailure hook
- **Feedback ROI:** 48 events tracking which user feedback led to improvements

### LEARNINGS.md — Severity-Tracked Rules

When a pattern reaches severity 3 (confirmed across multiple sessions), it gets promoted to `LEARNINGS.md` and `~/.claude/rules/learnings.md` (global). These are loaded into EVERY Claude Code session on this machine.

**Current severity-3 rules:**

- Anthropic key regex must include hyphens (`sk-[A-Za-z0-9\-]{20,}`)
- Citation integrity — never cite unverified academic sources
- Polybot prime directive — financial mission with three pillars
- Peak/off-peak budgeting — 40-50% peak, 100% off-peak
- 15-min crypto markets PERMANENTLY BANNED from live trading

### Practical Impact

The self-learning system has produced measurable improvements:

- **Mistake prevention:** Corrections from past sessions surface as warnings at init. You see "Edit: old_string not found — (3x) — avg fix: 12s" before you hit the same issue.
- **Strategy evolution:** Principle registry scores approaches across domains. What works for CCA session management also informs Kalshi strategy selection.
- **Session quality tracking:** `session_outcome_tracker.py` tracks 125+ sessions of historical data (backfilled from git log). Trend analysis shows where quality improved or degraded.
- **Research ROI:** `research_roi_resolver.py` tracks which research paths led to profitable outcomes vs dead ends, informing future research prioritization.

KALSHI COORDINATION

# Kalshi Cross-Chat Coordination

CCA is the R&D department. The Kalshi bot monitoring chat is the trading floor. Here's how they communicate.

### The Pipeline

**CCA → Kalshi** (via `~/.claude/cross-chat/CCA_TO_POLYBOT.md`):
→ Research findings with verified citations
→ Strategy improvements with specific function signatures
→ Guard improvements with thresholds and data fields
→ New edge proposals with structural basis + math + implementation path

**Kalshi → CCA** (via `~/.claude/cross-chat/POLYBOT_TO_CCA.md`):
→ Live trading data patterns (what's working, what's failing)
→ Guard discovery results (new IL guards, CUSUM alerts)
→ Edge candidates that need CCA research validation
→ Strategy performance data for CCA's self-learning pipeline

### Rules (Non-Negotiable)

- Every CCA session checks POLYBOT_TO_CCA.md for pending requests
- Kalshi main checks CCA_TO_POLYBOT.md every 3rd monitoring cycle ( 15 min)
- Deliveries must be IMMEDIATELY ACTIONABLE: specific functions, thresholds, data fields
- "Consider using HMM" is useless. "Build a 2-state HMM using hmmlearn, fit on 60-day rolling windows, switch to conservative mode when P(high-vol) > 0.7" is useful
- Stale comms (>48h with no exchange): flag and write proactive update
- CCA doesn't wait to be asked — delivers proactively when it finds improvements

### What CCA Has Delivered

CCA has written 86+ deliveries (UPDATE entries) to CCA_TO_POLYBOT.md, including:

- Loss reduction simulator (REQ-057)
- Monte Carlo sizing analysis with bounded-loss re-runs
- Correlated loss analyzer (REQ-054)
- Market diversification via HHI (REQ-055)
- Lopez de Prado AFML pipeline findings
- Bet advisor API using principle registry
- Daily sniper edge analysis
- Weather market expansion research
- $15-25/day strategy framework

### Self-Learning Mechanisms (Active in Kalshi)

These run automatically in the Kalshi bot:

- **Bayesian posterior:** Updates on every drift bet settlement
- **Auto-guards:** Fires at session start, discovers new guard buckets
- **CUSUM/SPRT:** Sequential testing for strategy drift detection
- **Page-Hinkley:** Change-point detection in performance streams

- **Strategy health scorer:** HEALTHY/MONITOR/PAUSE/KILL verdicts per strategy

CURRENT STATE

# Current State & Next Steps

### Where We Are (Session 246, April 1, 2026)

**Healthy**
- 10,706 tests passing (343/350 suites)
- 7 pre-existing failures (autoloop x2, references)
- All 3 custom agents validated
- 155K LOC across 9 modules
- 461 intelligence findings
- 1,226 self-learning events
- 18 live hooks running
- Zero external Python dependencies

**Needs Attention**
- PyBoy → mGBA migration for Pokemon bot
- Ebbinghaus decay needs schema integration
- SESSION_STATE.md is 38K tokens (needs pruning)
- CLAUDE.md is 11.4K tokens (58% reduction plan exists)
- 3 autoloop test failures (pre-existing)
- Post-promo rate limits are tighter
- Agent Teams: untested in production

### What's Planned Next

**Chat 15 (next session):**
- Build `cca-scout` agent (sonnet) — autonomous subreddit scanning agent
- Test-runner hardening — handle edge cases found during validation
- Ebbinghaus decay schema integration — add `last_accessed_at` to memory store

**Chat 16:**
- Agent Teams in `/cca-nuclear` — use native agents for parallel URL review
- SessionStart hook — initialize session state on first tool call
- SubagentStart budget controls — token limits per spawned agent

**Chat 17:**
- Compaction v2 — use CC source knowledge to patch the exact compaction bug
- Cross-chat delivery batch — process all pending POLYBOT_TO_CCA requests
- Phase 5 plan — next major sprint planning

### Master Task Status Summary

| Status | Count | Notable |
|---|---|---|
| Complete | 10 | MT-2, MT-3, MT-4, MT-6, MT-7, MT-20, MT-22, MT-26, MT-29, MT-37 |
| Active | 35 | MT-0, MT-1, MT-9, MT-10, MT-11, MT-14, MT-17, MT-21, MT-28, MT-32, MT-33, MT-49, MT-53… |
| Pending | 5 | MT-5, MT-34, MT-50, MT-51… |
| Blocked | 1 | MT-23 (Telegram — needs API key) |

> **The mission remains:** Get Smarter (self-learning, memory, intelligence) and Get More Bodies (automation, multi-chat, autoloop). Every session should leave the system measurably better than it found it. That's the measure — not "did I write code" but "is the ecosystem objectively smarter and more capable because of what was done today."

Generated by CCA Session 247 · April 1, 2026
github.com/mpshields96/ClaudeCodeAdvancements